

FPGA Accelerated Post-Quantum Cryptography

He Li [✉], *Member, IEEE*, Yongming Tang [✉], Zhiqiang Que [✉], *Member, IEEE*,
and Jiliang Zhang [✉], *Senior Member, IEEE*

Abstract—Recent advancement in quantum information processing technology has led to the emergence of advanced cryptography in the post-quantum era. Next generation cryptographic techniques aim to be mathematically resistant against any known attacks related to quantum computing, and can be easily implemented on traditional hardware platforms. The National Institutes of Standards and Technology (NIST) has entered the fourth-round standardization process of post-quantum cryptography (PQC). Software implementations of PQC candidates have been widely investigated. Interests in domain-specific hardware acceleration of PQC algorithms have risen, in particular using field-programmable gate arrays (FPGAs). While conventional general-purpose hardware platforms have been used for PQC implementations, modern FPGAs promise software-hardware co-optimisation, deep pipeline parallelism and trivial support for custom-precision arithmetic. Therefore, the time is ripe for reviewing recent FPGA-based PQC implementations. This article first surveys state-of-the-art advances in PQC implementations on FPGAs, including fast arithmetic, algorithm-hardware codesign approaches and open-source PQC hardware projects, then gives a brief review of recent attacks on PQC algorithms and their hardware implementations. Finally, we summarise the challenges for hardware implementations along with potential research directions in this promising field.

Index Terms—Post-quantum cryptography, FPGA, hardware-software codesign, hardware acceleration.

I. INTRODUCTION

SINCE quantum computers are expected to be practically available in the future decades, it is necessary for us to study cryptographic techniques preventing attacks from quantum computers. This area of research is called post-quantum cryptography [1]. In order to counter the quantum threat, the National Institute of Standards and Technology (NIST) has launched a process to select and standardize new resilient algorithms [2]. The open selection process started in 2017, with a first round of 69 proposed algorithms, which were presented, analysed and attacked by the crypto community. NIST has completed

the third round of the PQC standardization process, where four candidate algorithms have been selected for standardization and four additional algorithms to be evaluated in the fourth round.

The implementations for PQC schemes have been a very active research field in the past decade [3]. FPGA-tailored design methodology has been demonstrated to be efficient, parameterizable, and scalable for PQC implementations [4]. FPGAs promise deep pipeline parallelism, modern hardware architecture and fast custom-precision arithmetic operations. Therefore, the time is ripe for describing recent development of PQC accelerators on FPGAs.

There have been several surveys investigating recent advances of PQC in terms of algorithms [5], [6], implementations [4], [7] and applications [3]. For a specific PQC scheme, Balamurugan et al. reviewed the code-based cryptography and potential research directions that are yet to be explored [6]. Sendrier reviewed the principle of McEliece public-key encryption scheme and McEliece variants [5]. However, there are no surveys reviewing recent domain-specific accelerators on FPGAs for code-based PQC. We will fill this gap in Section II in this article. As another promising PQC scheme, lattice-based cryptography provides a rich set of primitives. In 2017, Nejatollahi et al. surveyed both software and hardware implementation challenges for lattice-based cryptography [4]. Recently, Xie et al. have presented an overall design flow for the high-level synthesis (HLS)-based implementation of the lattice-based PQC candidates [7]. However, they only briefly reviewed several NIST Round 2 PQC candidates which were published before 2019. This article hence reviews the state-of-the-art FPGA-based PQC implementations since then, including lattice-based, code-based and isogeny-based schemes.

In this article, we first survey state-of-the-art FPGA-based PQC implementations, particularly focusing on advanced architectural optimisations and open-source hardware projects. We then discuss recent attacks on PQC algorithms and some hardware implementations. Finally, we summarise research challenges and future research directions in PQC implementations. The novel contributions of this article are as follows.

- A survey of hardware implemented PQC schemes based on fast arithmetic, software-hardware codesign and high-level synthesis, with a particular focus on the state-of-the-art lattice-based, isogeny-based and code-based PQC schemes, as well as available open-source FPGA-based PQC accelerators.
- A review of recent attacks on both PQC algorithms and PQC hardware implementations, which facilitates the design of potential countermeasures against PQC attacks.

Manuscript received 18 August 2022; accepted 20 October 2022. Date of publication 28 October 2022; date of current version 15 November 2022. This work was supported by the National Natural Science Foundation of China under Grants 62122023 and U20A20202. The review of this article was arranged by the guest editors of the Special Issue for NANOARCH 2021. (*Corresponding author: Jiliang Zhang.*)

He Li and Yongming Tang are with the School of Electronic Science and Engineering, Southeast University, Nanjing 210096, China (e-mail: he.li@ieee.org; tym@seu.edu.cn).

Zhiqiang Que is with the Department of Computing, Imperial College London, London SW72BX, U.K. (e-mail: z.que@imperial.ac.uk).

Jiliang Zhang is with the College of Semiconductors (College of Integrated Circuits), Hunan University, Changsha 410082, China (e-mail: zhangjiliang@hnu.edu.cn).

Digital Object Identifier 10.1109/TNANO.2022.3217802

TABLE I
COMPARISON OF FPGA ACCELERATED LATTICE-BASED PQC IMPLEMENTATIONS

Scheme	Devices	Operations	LUTs	FFs	DSPs	BRAMs	$f_{\max}$ (MHz)	Runtime (μ s)	Throughput (Mbps)
NewHope [8]	Artix-7	Encryption	4635	4498	2	4	117	0.015	
R-LWE [9]	Kintex-7	Encryption	898	815	1	3	304	229	0.021
		Decryption	635	190	1	1	303	114	0.026
R-LWE [10]	Kintex-7	Encryption/ Decryption	1381	190	1	1	275	129 / 64.5	0.038 / 0.046
R-LWE [11]	Virtex-6	Encryption/ Decryption	1349	860	1	2	313	20 / 9	0.053 / 0.116
R-LWE [12]	Spartan-6	Encryption	-	-	16	247	3.72	0.292	
R-LWE (R2MDC) [19]	Stratix-4	Encryption	28105	28358	26	220	232	5.08	0.211
		Decryption	4122	6178	22	15	237	2.72	0.383
R-LWE (R2MDC) [13]	Virtex-7	Encryption	7334	5644	31	0	291	1.09	1
		Decryption		2549	1884	11	302	1.04	1
R-LWE (R4MDC) [13]	Virtex-7	Encryption	13358	9762	59	0	290	0.63	1.72
		Decryption	4632	3401	21	0	299	0.61	1,722
BRLWE [14]	Stratix V	Encryption/ Decryption	-	-	0	0	317	207	-
M-LWE (Kyber512) [15]	Artix-7	Encryption/ Decryption	442	237	1	3	136	52.92	0.020
M-LWR [16]	UltraScale+	Encapsulation	24.9k	10.7k	0	2	150	20.9	0.205
M-LWR [17]	UltraScale+	Encapsulation/ Decapsulation	12.3k	11.2k	256	3.5	322	43/	0.1/
								41	0.063

TABLE II
COMPARISON OF FPGA ACCELERATED ISOGENY-BASED PQC IMPLEMENTATIONS

Scheme	Devices	Mults	LUTs	FFs	Slices	DSPs	BRAMs	$f_{\max}$ (MHz)	Runtime (ms)
SIDHp503 [25]	Virtex-7	10	28520	39446	12962	320	34.5	176	19.8
SIDHp751 [25]	Virtex-7	10	40327	56979	18094	470	44	173	43
SIDHp503 [26]	Virtex-7	12	33969	45615	13203	384	40	207	14.2
SIDHp751 [26]	Virtex-7	12	50084	69054	19892	576	54.5	201	31.6
SIDHp1019 [26]	Virtex-7	12	67432	95895	26976	768	68	195	58.4
SIDHp1533 [26]	Virtex-7	12	104684	144127	40279	1152	85	193	130.3
SIKEp434 [27]	Virtex-7	8	-	-	7408	162	38	142	24.3
SIKEp503 [27]	Virtex-7	8	-	-	7408	162	38	142	28.7
SIKEp606 [27]	Virtex-7	8	-	-	7408	162	38	142	51.8
SIKEp751 [27]	Virtex-7	8	-	-	7408	162	38	142	60.8
SIKEp751 [28]	Virtex-7	3	49099	62124	18711	294	22.5	225	31.55
SIKEp771 [29]	Virtex-7	1	164390	153671	53856	486	-	156	-
SIKEp751/ SIDHp751 [30]	Virtex-7	8	20207	39339	11136	452	41.5	232	25.5
SIKEp751/ SIDHp751 [31]	Virtex-7	6	37502	54121	15246	456	54	182	18.2
SIKEp751/ SIDHp751 [32]	Virtex-7	1	90940	68895	27286	834	73.5	155	9.3

- Characteristic analyses of various PQC algorithms that are of benefit or hindrance to hardware implementations, highlighting research challenges addressing the latter, along with several promising avenues for future work.

II. STATE-OF-THE-ART PQC HARDWARE IMPLEMENTATIONS

We now review the state-of-the-art lattice-based, isogeny-based and code-based PQC implementations on FPGAs, and several publicly available open-source PQC implementations.

A. Hardware Implementation of Lattice-Based PQC Schemes

Lattice-based cryptography (LBC) has emerged as the most viable substitutes to the classical cryptographic schemes. Many hard mathematical problems underpin LBC schemes, such as a commonly used learning-with errors (LWE) problem. The LWE schemes use standard lattices, which require matrix-vector multiplications with high compute-source and memory cost due to their inherent quadratic complexity. As an alternative to standard lattices, the Ring-LWE (R-LWE) represents the matrix

of standard lattices by a single row in ring lattices. The remaining rows are generated by cyclic shifts of the first row. This leads to simpler calculations, because the main arithmetic operation, i.e., matrix-vector multiplication, is replaced with polynomial multiplication [13].

Kundi et al. explored novel architectural optimizations for polynomial multiplication to achieve an ultra-high throughput Ring-LWE implementation on FPGAs [13]. A completely parallel architecture of a schoolbook polynomial multiplication, and radix-2 and radix-4 pipelined NTT architectures were investigated. Results collected on a latest Xilinx UltraScale+ FPGA showed a $5\times$ speedup than its counterparts. Moreover, the proposed high-speed NTT takes only 0.63μ s for the encryption [13]. To explore the inherent approximate nature of LWE problem, Kundi et al. further presented a multi-level approximate Ring-LWE (AxRLWE) architecture in order to be implemented on lightweight IoT devices. Ring-LWE implementations following AxRLWE on FPGAs demonstrated 64% area reduction compared to earlier accurate Ring-LWE designs [13]. However, AxRLWE suffers from a reduced quantum security [18].

As a new variant of Ring-LWE, the binary Ring-Learning-with-Errors (BRLWE) scheme uses binary errors to achieve low-complexity implementation. Xie et al. presented efficient implementations of the finite-field arithmetic for BRLWE. The key arithmetic of the BRLWE scheme is implemented by a LUT-like method. The implemented BRLWE outperforms its fast-speed Ring-LWE counterparts with an up-to 16.2% reduction in area-delay product and 21.6% area reduction [19]. He et al. employed a hybrid field based polynomial multiplication for BRLWE implementations, which demonstrated a 66.0% less area-delay product than Ebrahimi et al.'s implementation [20] on a Straix V device [14].

In addition, NewHope is a key-exchange protocol based on the Ring-LWE problem. Beckwith and Diehl presented a pipelined implementation of NewHope512cpa with 652.2 Mbps, 0.088 Mbps/LUT (i.e., throughput-to-area ratio) and 0.79 nJ/bit energy efficiency. This demonstrated $10.05\times$ and $10.2\times$ improvements in throughput-to-area and energy efficiency, respectively, versus a non-pipelined approach [21].

CRYSTALS-Kyber is another quantum-resistant and promising LBC, which is based on the hardness of module-learning with errors (M-LWE). Yao et al. proposed a M-LWE cryptoprocessor targeting CRYSTALS-Kyber with area-time trade-off for the first time [22]. This balanced design considered a fast and low-cost binomial sampler and vector-polynomials multiplication structure based on a pipelined decimation-in-frequency (DIF) NTT technique. This M-LWE cryptoprocessor achieved 27,708 encryption operations per second using only 690 hardware slices and 106,716 decryption operations per second using only 571 hardware slices [22]. The variadic parameters allow M-LWE to obtain a more flexible security-performance trade-off than Ring-LWE.

Interest in applying HLS to accelerate lattice-based PQC algorithms on FPGAs has been increasingly growing. In 2019, Ozcan and Aysu first employed HLS to design PQC accelerators on a Xilinx Virtex-7 FPGA [23], where thirteen PQC candidates are benchmarked in their study (seven key encapsulation module, six signature schemes). The authors used HLS to perform a design space exploration for NTT comparing HLS implementations with corresponding hand-coded RTL designs [23]. Recently, Xie et al. have employed HLS to implement BRLWE scheme using Intel Quartus Prime 17.0 on a Stratix-V device [7]. The HLS-generated BRLWE outperforms the RTL implementation at least 45% reduction in area-delay product [7].

As for a hardware-software co-design approach, Mera et al. exploited the idea of distributed computing at micro-architectural level together with novel algorithmic optimizations to achieve approximately a $6\times$ speedup with respect to optimized software at a small area cost on a Zynq-7000 ARM/FPGA SoC [24]. The execution is performed on an ARM core and only the most computationally expensive operation, i.e., the polynomial multiplication, is offloaded to the co-processor to obtain a compact design [24]. Table 1 summarises comparisons of FPGA accelerated lattice-based PQC implementations.

B. Hardware Implementation of Isogeny-Based PQC Schemes

Compared with lattice-based PQC candidates, the main practical limitation of isogeny-based PQC lies in its performance

To address this issue, many hardware accelerations of the supersingular isogeny Diffie-Hellman (SIDH) key exchange and supersingular isogeny key encapsulation (SIKE) protocols have been implemented on FPGAs [32]. Koziel et al. proposed the first architecture of the SIDH protocol in 2016 [25]. Later in 2018, Koziel et al. implemented the first SIDH implementation close to the 256-bit quantum security level, demonstrating a $2\times$ speedup against performance-optimized software implementations [26]. In 2020, Koziel et al. presented a fast parallel architecture to accelerate SIKE on the Xilinx Artix-7, Virtex-7, and Kintex UltraScale+ FPGA families. At the NIST security level 5, the authors can execute the entire SIKE protocol in 15.3 μs on a Kintex UltraScale+ FPGA [33].

Roy and Mukhopadhyay developed area and memory efficient modules for implementing SIKE on FPGAs [28]. Since the most time-consuming operation is multiplication, an implementation of Montgomery multiplier based on a redundant number system was presented. The carry propagation during integer multiplication and multi-operand addition can then be restricted to small values. Elkhatib et al.'s SIKE implementation demonstrated a runtime drop from 33.4 ms to 25.5 ms for the SIKEp751 (NIST security level 5) based on a Virtex-7 FPGA [30]. Farzam et al. improved the modular multiplier [31] and obtained a speedup of about $1.4\times$ for SIKEp751 compared to [30]. Moreover, Tian et al. proposed a new constant-time field arithmetic logic unit (FALU), which is the core design of the SIKE architecture [32]. Their SIKE design only consumed 9.3 ms with $f_{max} = 155.8$ MHz, about $2\times$ faster than Farzam et al.'s proposal [31], where the modular multiplier merely costs 16 clock cycles [32]. In addition, multi-precision implementations of SIKE protocol have been investigated to achieve high throughput [29]. We remark that the aforementioned hardware implementations can be configured to target only one security level at a time. Masolino et al. proposed a hardware-software co-design method supporting on-the-fly operation to execute SIKE hardware for any current SIKE parameters equivalent to the post-quantum security of AES-128/192/256 and SHA3-256 [27]. Table 2 summarises comparisons of FPGA accelerated isogeny-based PQC implementations.

C. Hardware Implementation of Code-Based PQC Schemes

Code-based cryptography using binary Goppa codes is a promising solution for thwarting attacks based on quantum computers. The McEliece cryptosystem is a code-based public-key cryptosystem which is based on linear error-correcting codes. Even though the McEliece cryptosystem is believed to be resistant against quantum attacks, it is vulnerable to fault injection attacks [34]. Canto et al. proposed a novel error detection schemes derived for different Goppa arithmetic units where the McEliece cryptosystem is able to use multiplication, squaring, division, square root, inversion, and greatest common division [34]. López-García and Cantó-Navarro presented a McEliece cryptosystem for PQC providing larger than 128 bits of quantum security via a public key over 2 million bytes. The hardware-software codesign system used an ARM Cortex-A53 core connected to a coprocessor through an AXI4 lite interface. The complete system implemented on a Xilinx Zynq UltraScale+ FPGA is able to decipher texts of 8192 b-length in 47.39 ms [35].

D. Open-Source PQC Implementations on FPGAs

To increase the reproducibility and allow for unified comparison among different PQC accelerators, many research teams opensourced their FPGA implementations. Roy and Basso presented a generic implementation framework for lattice-based PQC schemes [17]. Their Vivado project and hardware description language (HDL) source codes are open-sourced.¹ Agrawal et al. [36] introduced a set of FPGA-based PQC primitives consisting of four frequently used security components, i.e., public key cryptosystem (PKC), key exchange (KEX), oblivious transfer (OT), and zero-knowledge proof (ZKP). To address scalability and efficiency issues, a set of highly-optimized, parameterizable hardware modules were introduced to serve as primitives for faster design space exploration of post-quantum cryptosystems, especially the systems using Ring-LWE algorithms. An open-sourced RTL library,² is fully parameterizable with an efficient, n -point NTT based high speed polynomial multiplier. Massolino et al. demonstrated that supersingular isogeny-based primitives such as SIDH and SIKE can indeed be deployed on FPGAs for embedded applications featuring competitive performance [27], where the entire project is publicly available³.

III. ATTACK ANALYSES FOR PQC

A. Attacks on PQC Algorithms

Recently, two novel attacks were reported on Rainbow and SIDH protocols. Beullens introduced a key recovery attack against the Rainbow signature scheme. Given a Rainbow public key for the security level 1 parameters of the second-round submission, the proposed attack returns the corresponding secret key after on average 53 hours of computation time on a laptop with 8-core Intel i9-10885H CPU, running at 2.5 GHz [37]. In addition, Castryck and Decru recently presented an efficient key recovery attack on the SIDH protocol, based on the “glue-and-split” theorem [38]. This attack exploited the existence of a small non-scalar endomorphism on the starting curve and the auxiliary torsion point information that Alice and Bob are shared during the protocol [38]. The experimental results showed that it took about one hour running on Intel Xeon CPU E5-2630v2 at 2.60 GHz to break SIKEp434 [38]. The validity of these attacks was quickly acknowledged and will probably lead to the abandonment of these algorithms. Therefore, open disclosure of the details of the algorithms is crucial to enable the community to analyse them and find possible flaws.

B. Attacks on PQC Hardware Implementations

Many well-known cryptographic algorithms, such as AES and RSA, have been vulnerable to side channel attacks and hardware Trojans [39]. Identification of the security vulnerabilities in PQC algorithms, as well as the compromises that must be made

to achieve that level of protection, are crucial for the PQC standardisation process.

Typical side channel attack methods include cold boot attack, fault attack, timing attack and power analysis. Attacks using the cold boot method need direct physical access to the target device. In this attack, an attacker uses coolant to freeze the DRAM. The DRAM data is subsequently copied to an external device for additional analysis. There has been evidence of a generic attack on the NTT, which is frequently employed by ring- and module-LWE PQC algorithms such as Kyber and NewHope [40]. This attack acquired at least 60% of the secret coefficients.

The fault attack is another popular technique for breaking into cryptographic systems and obtaining sensitive data. This attack introduces unexpected operations to leak critical data that can be used to recover the key [39]. By examining the length of time needed to process cryptographic algorithms and exploiting this information to recover secret information, timing attacks are introduced [41]. Implementing the algorithms in constant time is a key defence against this attack, but resulting in high overhead. Timing attacks against numerous PQC algorithms, including FrodoKEM, HQC and Falcon [41], have been discovered.

Simple power analysis (SPA) and differential power analysis (DPA) have also been used to attack PQC implementations [42], [43]. SPA requires a direct analysis of observed power or electromagnetic (EM) radiation traces [42]. DPA needs additional analysis, such as statistical correlation or device profiling and templating with in-depth knowledge of the inner workings of the algorithm [43].

IV. CHALLENGES AND FUTURE DIRECTIONS

A. Challenges

During the initial round of the NIST evaluation process, the primary attention was focused on PQC security analysis, where reference software implementations existed for the majority of the candidates. Thereafter, optimized software implementations taking advantage of vector instructions of modern microprocessors were considered. In the second and third rounds, software-hardware co-design and hardware acceleration of PQC algorithms have been reported. Some challenges still exist for efficient implementations of PQC algorithms.

- Complexity of PQC algorithms: PQC algorithms have been originally designed for the demonstration of their security impact. Hardware designers require necessary mathematical knowledge, such as number theory, abstract algebra, coding theory, to understand the specifications of PQC algorithms. Hence, mapping a complex mathematical description to a executable software (e.g. C or C++) or hardware (e.g. Verilog) codes is a challenge.
- Various PQC families: popular post-quantum cryptography algorithms include lattice-based, code-based and isogeny-based cryptography [5], [44]. Each has several subfamilies, which requires different theoretical background, resulting in different sets of basic operations and optimization strategies.

¹[Online]. Available: https://github.com/sujoyetc/SABER_HW

²[Online]. Available: <https://ascslab.org/research/pqcp/index.html>

³[Online]. Available: <https://github.com/pmssolino/hw-sike>

- Lack of open-source hardware implementations: there have been some open-source software implementations [3], however, only a few open-source hardware contributions exist, as have been summarised in Section II.D, which would require major driving improvements in the open hardware community [45]. Moreover, hardware implementations also lack well defined evaluation metrics for comparisons due to various implementation platforms and optimization targets. Therefore, a unified design space evaluation framework in terms of area, power, throughput and latency is also required.
- Memory optimisation: the storage of public keys, private keys and internal states requires a large amount of memory resource in hardware [6]. This would prohibit lightweight and efficient hardware implementations of PQC algorithms on FPGAs.
- Trustworthy interconnections: even though PQC algorithms are designed to protect critical systems from attacks using quantum computers. From a system perspective, PQC hardware will be connected to other devices or communication protocols, most of which can be untrusted. It is expected that additional protection methods combined with PQC will be needed to prohibit side-channel [46], key leakage or manipulation attacks [47].

B. Emerging Techniques for PQC Hardware Design

1) *Fast Arithmetic Circuits*: Implementations of PQC algorithms on FPGAs require many arithmetic operators, such as multiplication [9], polynomial multiplication [48], multiply-accumulate [49] and customised NTT cores [50]. For example, the most time-consuming operations in lattice-based cryptography is the polynomial multiplication, which can be performed by using SPMA and NTT. Appropriate arithmetic implementations can produce higher speed and maintain balanced area-time efficiency. This is especially useful for edge computing devices or medium-size FPGAs [50]. Some operations in various PQC algorithms have the potential for high degree of parallelization, which can be optimised by parallel arithmetic implementations based on modern hardware architectures.

2) *HLS Optimisations*: Coding at the register transfer level (RTL) is extremely time-consuming, requiring months of development and verification. HLS offers a promising solution to cover this gap, accepting C/C++ as a design entry. Nevertheless, the Quality-of-Results (QoR) achievable using HLS can be lower compared to hand-made RTL solutions, due to the difficulties of the synthesis process in handling general-purpose software. For this reason HLS finds its place as prototyping platform or for doing design space exploration. For the NIST submissions all the algorithms have been implemented in C, a language prone for HLS on FPGAs. The adoption of HLS in PQC designs can be extremely interesting for bridging the gap from algorithm to hardware design, therefore reducing time-to-market. Moreover, starting from the same source code, HLS is capable of producing a wide range of different microarchitectures, each

with different characteristics in terms of latency and resource utilization depending on the optimisation directives adopted. It is expected that more investigations in HLS will be reported to generate PQC algorithms in the near future.

3) *Other Hardware Design Optimisations*: Potential novel design methods can be investigated for efficient PQC hardware accelerators:

- Low power architecture: one of the widely discussed PQC application domain is related to Internet-of-Things (IoT) devices. A low-power PQC module is needed to be performed in the remote devices with limited power supply ability.
- EDA for PQC: due to the high algorithmic complexity in most PQC families, design of user-friendly EDA tools for PQC design is also necessary, thereby allowing PQC to facilitate security protection in many critical computing and communication systems.
- PQC fast evaluation method: since there have been several PQC candidates still in the NIST competition, and also existing PQC algorithms will be upgraded, a systematic FPGA design flow for PQC evaluations is highly required to investigate performance, power, area trade-offs for different PQC families.

4) *Quantum-Classical Codesign*: Apart from PQC itself, it would be possible to add another layer of protection, such as using quantum key distribution (QKD) and quantum random number generation (QRNG). QRNG can be used for all key generation processes. QKD and nowadays QKD networks can also be applied for specific applications, such as long-term protection of communication backbones. The future of cybersecurity will only be achieved by following a quantum-classical codesign principle [51].

C. PQC Applications

Successful post-quantum cryptosystems can lead to enhance the security of a number of relevant fields whose applications rely heavily on resource-constrained and battery-dependent devices or high bit rates.

1) *PQC for 5G*: FPGAs promise an ideal hardware platform for 5G applications, because they provide hardware support for multiple signaling standards, multiple bands and multiple sub-networks. FPGAs, therefore, enable large diversity in 5G applications. Given that PQC algorithms are aiming at high bit rates, a high-throughput architecture can be achieved producing one key per clock cycle. Therefore, fast PQC implementations can be incorporated into 5G applications to prevent potential attacks related to quantum computing.

2) *PQC With Quantum Networks*: Even though quantum-resistant key establishment has been traditionally performed through classical computational methods, recent development of QKD networks provides a hybrid security architecture, where secure keys established in QKD networks can be shared by using PQC-based equipment.

3) *PQC in IoT*: Novel architectures and implementations of different PQC schemes will be needed for testing and

validating energy-efficient quantum-resistant algorithms for resource-constrained IoT devices. Such post-quantum IoT architectures will be designed for home automation, smart buildings, smart factories and even smart cities.

D. Hardware Security for PQC Implementations

When PQC is used to prohibit confidential data stored either in future quantum computers or quantum networks from being hacked by malicious parties, necessary security issues should be investigated to ensure information confidentiality and integrity [1]. Analyses of PQC security vulnerabilities and countermeasures are necessarily needed in this area.

1) *Vulnerabilities from EDA Tools*: Even though PQC is immune to attacks from quantum computers, there is an underlying risk for PQC implementations, as they would be hacked by untrusted third-party IP cores or EDA tools. Therefore, security enhancement and trust verification methods are necessarily needed for trustworthy PQC implementations.

2) *Hardware Trojan in PQC Implementations*: Owing to the algorithm complexity in PQC schemes, potential attackers can insert hardware Trojans (HTs) at the design stage. Investigation of how and where to insert HTs in PQC implementations will provide valuable insights for efficient HT detection. Since hand-written RTL, HLS, hardware-software codesign methods have been used for PQC designs, promising HT prevention and detection approaches at the design stage are demanded to avoid and isolate HTs.

3) *Countermeasures against Side-Channel Attacks*: Novel countermeasures to prevent PQC implementations from side-channel attacks are highly needed in the future. Promising strategies fall into two main categories: (i) reduce or even eliminate the release of side-channel information, and (ii) make the leaked information uncorrelated to the secret data. In the first category, a low-cost power conditioning and filtering technique can be used to prevent monitoring the power of PQC circuits. We can also jam the emitted channel with noise, such as a random delay to deter timing attacks. In the second category, the masking countermeasure is a promising technique in PQC implementation. For example, we can manipulate different subsets of keys via XOR operations, devoid of manipulating the sensitive keys as a whole.

V. CONCLUSION

In this article we presented a comprehensive survey for recent methodology to optimize various PQC designs on FPGAs, providing detailed technical and implementational advances. To demonstrate the opportunity of using reconfigurable hardware platforms for this new class of applications, fast arithmetic units, architectural optimisations, hardware-software codesign methodology, high-level design techniques and open-source FPGA-implemented PQC projects are comprehensively analysed. Discussions on attacks and countermeasures for PQC hardware implementations are also provided. We remark that this article collects and presents concrete implementations, showing modern FPGA design techniques can be pragmatically and effectively applied. It is a foreseeable advantage for making PQC hardware designers cognizant when designing with FPGAs, to

profit of emerging architectures and design tools with negligible effort and in a short time.

REFERENCES

- [1] D. J. Bernstein and T. Lange, "Post-quantum cryptography," *Nature*, vol. 549, pp. 188–194, 2017.
- [2] "NIST PQC standardization process," 2022. [Online]. Available: <https://csrc.nist.gov/Projects/post-quantum-cryptography>
- [3] T. M. Fernández-Caramés, "From pre-quantum to post-quantum IoT security: A survey on quantum-resistant cryptosystems for the Internet of Things," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6457–6480, Jul. 2020.
- [4] H. Nejatollahi et al., "Post-quantum lattice-based cryptography implementations: A survey," *ACM Comput. Surv.*, vol. 51, pp. 1–41, 2019.
- [5] N. Sendrier, "Code-based cryptography: State of the art and perspectives," *IEEE Secur. Privacy*, vol. 15, no. 4, pp. 44–50, 2017.
- [6] C. Balamurugan et al., "Post-quantum and code-based cryptography—Some prospective research directions," *Cryptogr.*, vol. 5, no. 4, 2021, Art. no. 38.
- [7] J. Xie, K. Basu, K. Gaj, and U. Guin, "Special session: The recent advance in hardware implementation of post-quantum cryptography," in *Proc. IEEE VLSI Test Symp.*, 2020, pp. 1–10.
- [8] T. Oder and T. Güneysu, "Implementing the newhope-simple key exchange on low-cost FPGAs," in *Proc. Int. Conf. Cryptol. Inf. Secur. Latin Amer.*, 2017, pp. 128–142.
- [9] W. Liu, S. Fan, A. Khalid, C. Rafferty, and M. O'Neill, "Optimized schoolbook polynomial multiplication for compact lattice-based cryptography on FPGA," *IEEE Trans. Very Large Scale Integr. Syst.*, vol. 27, no. 10, pp. 2459–2463, Oct. 2019.
- [10] Y. Zhang, C. Wang, D. E. S. Kundi, A. Khalid, M. O'Neill, and W. Liu, "An efficient and parallel R-LWE cryptoprocessor," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 67, no. 5, pp. 886–890, May 2020.
- [11] S. S. Roy et al., "Compact ring-LWE cryptoprocessor," in *Proc. Proc. Int. Workshop Cryptographic Hardware Embedded Syst.*, 2014, pp. 371–391.
- [12] C. Du et al., "High-speed polynomial multiplier architecture for ring-LWE based public key cryptosystems," in *Proc. ACM Int. Great Lakes Symp. VLSI*, 2016, pp. 9–14.
- [13] D.-E.-S. Kundi, Y. Zhang, C. Wang, A. Khalid, M. O'Neill, and W. Liu, "Ultra high-speed polynomial multiplications for lattice-based cryptography on FPGAs," *IEEE Trans. Emerg. Topics Comput.*, early access, Jan. 25, 2022, doi: [10.1109/TETC.2022.3144101](https://doi.org/10.1109/TETC.2022.3144101).
- [14] P. He, U. Guin, and J. Xie, "Novel low-complexity polynomial multiplication over hybrid fields for efficient implementation of binary ring-LWE post-quantum cryptography," *IEEE Trans. Emerg. Sel. Topics Circuits Syst.*, vol. 11, no. 2, pp. 383–394, Jun. 2021.
- [15] Z. Chen, Y. Ma, T. Chen, J. Lin, and J. Jing, "Towards efficient kyber on FPGAs: A processor for vector of polynomials," in *Proc. IEEE Asia South Pacific Des. Automat. Conf.*, 2020, pp. 247–252.
- [16] V. B. Dang et al., "Implementing and benchmarking three lattice-based post-quantum cryptography algorithms using software/hardware codesign," in *Proc. Int. Conf. Field-Programmable Technol.*, 2019, pp. 206–214.
- [17] S. S. Roy and A. Basso, "High-speed instruction-set coprocessor for lattice-based key encapsulation mechanism: Saber in hardware," *Proc. IACR Trans. Cryptographic Hardware Embedded Syst.*, 2020, pp. 443–466.
- [18] D.-E.-S. Kundi, A. Khalid, S. Bian, C. Wang, M. O'Neill, and W. Liu, "AxlWE: A multi-level approximate ring-LWE co-processor for lightweight IoT applications," *IEEE Internet Things J.*, vol. 9, no. 13, pp. 10492–10501, Jul. 2022.
- [19] J. Xie, P. He, and W. Wen, "Efficient implementation of finite field arithmetic for binary ring-LWE post-quantum cryptography through a novel lookup-table-like method," in *Proc. ACM/IEEE Des. Automat. Conf.*, 2021, pp. 1279–1284.
- [20] S. Ebrahimi, S. Bayat-Sarmadi, and H. Mosanaei-Boorani, "Post-quantum cryptoprocessors optimized for edge and resource-constrained devices in IoT," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 5500–5507, Jun. 2019.
- [21] L. Beckwith and W. Diehl, "New directions for newhope: Improving performance of post-quantum cryptography through algorithm-level pipelining," in *Proc. IEEE Int. Conf. Field-Programmable Technol.*, 2020, pp. 120–128.
- [22] K. Yao et al., "Towards crystals-kyber: A m-lwe cryptoprocessor with area-time trade-off," in *Proc. IEEE Int. Symp. Circuits Syst.*, 2021, pp. 1–5.

- [23] E. Ozcan and A. Aysu, "High-level synthesis of number-theoretic transform: A case study for future cryptosystems," *IEEE Embedded Syst. Lett.*, vol. 12, no. 4, pp. 133–136, Dec., 2020.
- [24] J. M. B. Mera, F. Turan, A. Karmakar, S. S. Roy, and I. Verbauwhede, "Compact domain-specific co-processor for accelerating module lattice-based kem," in *Proc. IEEE/ACM Des. Automat. Conf.*, 2020, pp. 1–6.
- [25] B. Koziel et al., "Fast hardware architectures for supersingular isogeny Diffie-Hellman key exchange on FPGA," in *Proc. Int. Conf. Cryptol. India*, 2016, pp. 191–206.
- [26] B. Koziel, R. Azarderakhsh, and M. M. Kermani, "A high-performance and scalable hardware architecture for isogeny-based cryptography," *IEEE Trans. Comput.*, vol. 67, no. 11, pp. 1594–1609, Nov. 2018.
- [27] P. M. C. Massolino et al., "A compact and scalable hardware/software co-design of SIKE," in *Proc. IACR Trans. Cryptographic Hardware Embedded Syst.*, vol. 2020, no. 2, 2020, pp. 245–271.
- [28] D. B. Roy and D. Mukhopadhyay, "Post quantum ECC on FPGA platform," *Cryptol. ePrint Arch.*, 2019.
- [29] J. Tian, J. Lin, and Z. Wang, "Fast modular multipliers for supersingular isogeny-based post-quantum cryptography," *IEEE Trans. Very Large Scale Integr. Syst.*, vol. 29, no. 2, pp. 359–371, Feb. 2020.
- [30] R. Elkhatib, R. Azarderakhsh, and M. Mozaffari-Kermani, "Highly optimized montgomery multiplier for SIKE primes on FPGA," in *Proc. IEEE Symp. Comput. Arithmetic*, 2020, pp. 64–71.
- [31] M. -H. Farzam, S. Bayat-Sarmadi, H. Mosanaei-Boorani, and A. Alivand, "Hardware architecture for supersingular isogeny Diffie-Hellman and key encapsulation using a fast montgomery multiplier," *IEEE Trans. Circuits Syst. I: Regular Papers*, vol. 68, no. 5, pp. 2042–2050, May 2021.
- [32] J. Tian, B. Wu, and Z. Wang, "High-speed FPGA implementation of SIKE based on an ultra-low-latency modular multiplier," *IEEE Trans. Circuits Syst. I: Regular Papers*, vol. 68, no. 9, pp. 3719–3731, Sep. 2021.
- [33] B. Koziel, A. -B. Ackie, R. E. Khatib, R. Azarderakhsh, and M. M. Kermani, "SIKE'd up: Fast hardware architectures for supersingular isogeny key encapsulation," *IEEE Trans. Circuits Syst. I: Regular Papers*, vol. 67, no. 12, pp. 4842–4854, Dec. 2020.
- [34] A. C. Canto, M. M. Kermani, and R. Azarderakhsh, "Reliable architectures for composite-field-oriented constructions of mceliece post-quantum cryptography on FPGA," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 40, no. 5, pp. 999–1003, May 2021.
- [35] M. López-Garcá and E. Cantó-Navarro, "Hardware-software implementation of a mceliece cryptosystem for post-quantum cryptography," in *Proc. Future Inf. Commun. Conf.*, 2020, pp. 814–825.
- [36] R. Agrawal, L. Bu, A. Ehret, and M. Kinsky, "Open-source FPGA implementation of post-quantum cryptographic hardware primitives," in *Proc. IEEE Int. Conf. Field Programmable Log. Appl.*, 2019, pp. 211–217.
- [37] W. Beullens, "Breaking rainbow takes a weekend on a laptop," *Cryptol. ePrint Arch.*, 2022.
- [38] W. Castryck and T. Decru, "An efficient key recovery attack on SIDH (preliminary version)," *Cryptol. ePrint Arch.*, 2022.
- [39] T. Kamucheka et al., "Power-based side channel attack analysis on PQC algorithms," *Cryptol. ePrint Arch.*, 2021.
- [40] M. Albrecht, A. Deo, and K. Paterson, "Cold boot attacks on ring and module LWE keys under the NTT," *IACR Trans. Cryptographic Hardware Embedded Syst.*, vol. 2018, no. 3, pp. 173–213, 2018.
- [41] J.-P. D'Anvers et al., "Timing attacks on error correcting codes in post-quantum schemes," in *Proc. ACM Workshop Theory Implementation Secur. Workshop*, 2019, pp. 2–9.
- [42] Z. Xu, O. Pemberton, S. S. Roy, D. Oswald, W. Yao, and Z. Zheng, "Magnifying side-channel leakage of lattice-based cryptosystems with chosen ciphertexts: The case study of kyber," *IEEE Trans. Comput.*, vol. 71, no. 9, pp. 2163–2176, Sep. 2022.
- [43] F. Aydin et al., "Horizontal side-channel vulnerabilities of post-quantum key exchange and encapsulation protocols," *ACM Trans. Embedded Comput. Syst.*, vol. 20, pp. 1–22, 2021.
- [44] J. Buchmann, K. Lauter, and M. Mosca, "Postquantum cryptography-state of the art," *IEEE Secur. Privacy*, vol. 15, no. 4, pp. 12–13, 2017.
- [45] E. Homsirikamol and K. G. George, "Toward a new HLS-based methodology for FPGA benchmarking of candidates in cryptographic competitions: The CAESAR contest case study," in *Proc. Int. Conf. Field Programmable Technol.*, 2017, pp. 120–127.
- [46] D. Apon and J. Howe, "Attacks on NIST PQC 3rd round candidates," in *Proc. IACR Real World Crypto Symp.*, 2021, pp. 737–743.
- [47] K. Bürstinghaus-Steinbach et al., "Post-quantum tls on embedded systems: Integrating and evaluating kyber and sphincs with mbed tls," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2020, pp. 841–852.
- [48] W. Tan, B. M. Case, A. Wang, S. Gao, and Y. Lao, "High-speed modular multiplier for lattice-based cryptosystems," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 68, no. 8, pp. 2927–2931, Aug. 2021.
- [49] J. Howe, C. Moore, M. O'Neill, F. Regazzoni, T. Güneysu, and K. Beeden, "Lattice-based encryption over standard lattices in hardware," in *Proc. IEEE/ACM Des. Automat. Conf.*, 2016, pp. 1–6.
- [50] Z.-Y. Wong, D. C.-K. Wong, W.-K. Lee, and K.-M. Mok, "High-speed RLWE-oriented polynomial multiplier utilizing karatsuba algorithm," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 68, no. 6, pp. 2157–2161, Jun. 2021.
- [51] H. Li, A. Wonfor, A. Weerasinghe, M. Alhussein, Y. Gong, and R. Pentty, "Quantum key distribution post-processing: A heterogeneous computing perspective," in *Proc. IEEE Int. System-on-Chip Conf.*, 2022, pp. 1–6.